

Histogram Equalization on the JVM — Executive Performance Analysis

Course: SISMD — Sistemas Multinúcleo e Distribuídos (ISEP)

Project: Histogram equalization, sequential vs. four concurrent strategies, with GC tuning

Repository: <https://github.com/Af-Oliveira/SISMD2026>

Project board: <https://github.com/users/Af-Oliveira/projects/6/views/1>

Date: April 2026

Table of contents

1. [Introduction](#)
 2. [Objectives](#)
 3. [Implementation approaches](#)
 4. [Concurrency and synchronization analysis](#)
 5. [Performance analysis](#)
 6. [Discussion](#)
 7. [Conclusions](#)
-

1. Introduction

Histogram equalization is a contrast-enhancement technique that re-maps every pixel's luminosity through the cumulative distribution function (CDF) of the input image's luminosity histogram. Dark images become brighter, washed-out images regain contrast, and the pixel intensity histogram of the output is — by construction — close to uniform. The algorithm is canonical in image processing courses and is also a textbook fit for parallelism: a 1920×1080 image contains ~2.07 million pixels, every pixel is independent, and the only data dependency is a tiny 256-entry histogram that has to be reduced before the rewriting phase can run.

This project implements **five** versions of the same algorithm and benchmarks them under **four** different garbage collectors, on a sweep of image sizes (640×480, 1280×720, 1920×1080) and thread counts ({1, 2, 4, 8, 16, 24}). The goal is empirical — to compare sequential, manual-threads, thread-pool, fork/join and Completable Future approaches on a real workload, and to quantify the effect of GC choice on the same workload.

2. Objectives

- Implement the five canonical execution strategies behind a single `ImageProcessor` interface so they are interchangeable.
- Verify that all five produce **bit-identical output** to the sequential baseline (correctness $\geq$ performance).
- Build a reproducible benchmark harness that captures execution time, heap usage and GC behavior per configuration.
- Tune the JVM by sweeping the workload under four garbage collectors and document the trade-offs.
- Produce an analysis that quantifies efficiency, scalability, overhead and bottlenecks.

3. Implementation approaches

The five processors are implemented within the `pt.isep.sismd.histogram` package and partition the algorithm into three identical stages, thereby ensuring that the comparison is conducted under equivalent conditions:

Stage 1: luminosity histogram (per-pixel read) — based on per-pixel read operations and inherently parallelizable.

Stage 2: cumulative histogram (256 adds, prefix) — which exhibits strict sequential data dependencies.

Stage 3: pixel rewriting (per-pixel write) — consisting of independent per-pixel write operations, enabling complete data-parallel execution.

The shared luminosity formula is the Rec. 601: $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$

```
public static int computeLuminosity(int r, int g, int b) {  
    return (int) Math.round(0.299 * r + 0.587 * g + 0.114 * b);  
}
```

3.1 Sequential baseline

The `SequentialHistogramEqualizer` executes the complete three-stage pipeline in a strictly single-threaded manner, without the use of synchronization primitives. Its memory requirements are minimal, consisting solely of the output array and two fixed-size histograms (256 integer bins each). This implementation is adopted as the reference baseline for functional correctness and as the denominator in speedup computations for comparative performance analysis.

3.2 Manual threads (Issue #4)

`ManualThreadHistogramEqualizer` partitions the input image by rows across a fixed number of worker threads (`numThreads`). Each worker maintains a private histogram (`int[256]`), thereby avoiding contention during the accumulation phase. Upon completion of the worker threads (after `join()`), the main thread performs a sequential reduction to merge the partial histograms into a single global histogram.

In Stage 3, the output image is partitioned using the same row-wise strategy, with each thread writing to a disjoint region of the output array. As a result, no synchronization mechanisms are required during this phase.

Synchronization design choice. The use of thread-local partial histograms followed by a sequential merge avoids the contention that would arise from fine-grained atomic updates (e.g., a 256-element `AtomicIntegerArray`, which would introduce per-bin contention and frequent compare-and-swap operations). Instead, each worker accumulates counts independently, and synchronization is deferred until thread completion via `join()`.

3.3 Thread pool (Issue #5)

`ThreadPoolHistogramEqualizer` preserves the same logical decomposition of the algorithm while delegating execution to a fixed-size `ExecutorService`. Each task, implemented as a `Callable<int[]>`, processes a distinct row-slice and returns a private partial histogram.

The main thread subsequently performs a sequential reduction over the returned partial histograms obtained from the corresponding `Future` objects. This aggregation step is computationally inexpensive, requiring only $256 \times N$ integer additions, where N is the number of tasks.

The thread pool is explicitly terminated within a `finally` block using a bounded shutdown timeout (30 seconds), ensuring deterministic resource release and making lifecycle management failures observable.

3.4 Fork/Join (Issue #6)

`ForkJoinHistogramEqualizer` employs a recursive divide-and-conquer strategy to decompose the workload. The input image is partitioned along the row dimension into progressively smaller subproblems until a predefined threshold (`default threshold = 50` rows) is reached, at which point computation is performed directly.

Partial results are propagated upward through the recursion tree via `RecursiveTask#join()`, with sibling results being merged during the return phase of the computation. The execution is managed by the JDK's `ForkJoinPool.commonPool()`, which eliminates the need for explicit pool lifecycle management. This represents a key advantage of the fork/join framework compared to the manually managed `ExecutorService` approach described in Section 3.3.

```
int mid = startRow + (rows >>> 1);
HistogramTask left = new HistogramTask(image, startRow, mid, threshold);
HistogramTask right = new HistogramTask(image, mid, endRow, threshold);
left.fork();           // schedule left asynchronously
int[] rightHist = right.compute(); // run right inline
int[] leftHist = left.join();    // wait for left
return mergeHistograms(leftHist, rightHist);
```

3.5 CompletableFuture (Issue #7)

`CompletableFutureHistogramEqualizer` models the computation as an asynchronous dataflow pipeline based on `CompletableFuture`. In Stage 1, the computation is expressed as a fan-out of `supplyAsync` tasks, each operating on a distinct row-slice and producing a partial histogram.

The resulting futures are then combined through a tree-like reduction using `thenCombine`, effectively performing pairwise aggregation of partial histograms until a single consolidated result is obtained.

In Stage 3, pixel rewriting is expressed as a set of `runAsync` operations, coordinated via `allOf`, where each task operates on disjoint output regions. This guarantees independence between tasks and eliminates the need for explicit synchronization during the write phase.

The dedicated `ForkJoinPool` is shut down deterministically in a `finally` block, the symmetric counterpart to the `ExecutorService` discipline of section 3.3.

3.6 Garbage Collector tuning (Issue #9)

The benchmark harness was run unchanged under four collectors:

GC	Flag	Family
Serial	<code>-XX:+UseSerialGC</code>	Single-threaded, stop-the-world
Parallel	<code>-XX:+UseParallelGC</code>	Multi-threaded throughput
G1	<code>-XX:+UseG1GC</code>	Region-based, pause-time-targeted (JDK 9+ default)
ZGC	<code>-XX:+UseZGC</code>	Concurrent, sub-millisecond pauses

Heap was pinned at `-Xms2g` for every run, everything else about the sweep was held constant. Detailed methodology and per-GC log evidence: `@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\gc-tuning\README.md`.

4. Concurrency and synchronization analysis

4.1 Work decomposition

All four parallel strategies partition the workload along the **row dimension** rather than at the granularity of individual pixels. This design choice is motivated by both memory layout considerations and concurrency efficiency.

- Java multidimensional arrays are stored in **row-major order**, meaning that a row slice corresponds to a contiguous region of memory. This improves spatial locality and allows hardware prefetchers to operate effectively.
- Row-level partitioning provides a sufficiently coarse-grained unit of work such that task scheduling and dispatch overhead is amortised across a large number of pixel operations per task.
- In Stage 3, row-wise partitioning yields disjoint regions of the output buffer, enabling lock-free writes since each thread operates on an exclusive segment of the array without interference.

4.2 The three sync points

Synchronization point	Timing	Mechanism
Histogram reduction	End of Stage 1	Each implementation produces per-worker partial histograms (<code>int[256]</code>), which are subsequently aggregated by the main thread (or by the parent frame in fork/join-based designs). This aggregation is performed without explicit locking.
Stage barrier 1 → 2	Between histogram construction and cumulative distribution function (CDF) computation	Synchronisation is achieved via task completion primitives (<code>join()</code> , <code>Future.get()</code> , or <code>thenCombine</code>). These constructs establish a <i>happens-before</i> relationship between worker thread writes and the main thread's visibility of the merged histogram.
Stage barrier 2 → 3	Between CDF computation and pixel rewriting	The same completion-based synchronisation mechanisms are used. The CDF array (<code>int[256]</code>) is treated as read-only during Stage 3, and safe publication is guaranteed through the completion barrier and Java Memory Model visibility guarantees.

The cumulative histogram computation (256 prefix-sum additions) is intentionally kept sequential across all parallel implementations. The overhead of parallelisation at this granularity would exceed its computational benefit due to task scheduling and coordination costs.

4.3 Race-condition strategy

There are exactly **zero** race conditions across the four parallel implementations. The strategy is uniform:

1. Workers write **only** to thread-local buffers.
2. Reductions happen **after** worker termination (via `join`, `Future.get`, `allOf`, or fork/join's parent-frame merge).
3. The output array is written by **disjoint slices** in stage 3.

This is verified empirically by the `HistogramEqualizationCorrectnessTest` suite (20 tests, including pixel-by-pixel equality with the sequential baseline on three image fixtures plus pathological inputs like single-color and two-tone images).

4.4 Why fork/join wins

- **Work-stealing.** Idle worker threads dynamically acquire tasks from deques associated with busier threads, enabling implicit load balancing. This mechanism mitigates potential imbalance in execution cost across row-slices, which is uncommon in this workload but improves robustness in heterogeneous cases.

- **Recursive merging.** In the fork/join model, histogram reduction is performed through a hierarchical, pairwise combination of partial results along the recursion tree. This yields a reduction depth of $O(\log N)$, as opposed to a flat $O(N)$ sequential aggregation performed on the main thread. With a pool size of 24 threads, this structure reduces aggregation overhead, particularly in small-to-medium input scenarios where reduction cost is non-negligible relative to computation.
- **Implicit pool management.** Execution is delegated to the `ForkJoinPool.commonPool()`, which is managed by the JVM. This avoids explicit thread pool instantiation and shutdown costs, eliminating associated lifecycle overhead and simplifying resource management.

5. Performance analysis

5.1 Methodology

The experimental platform consists of a **24-core system** running **JDK 26.0.1+8-34** on Windows, with a fixed **2 GB heap allocation**. Each configuration is executed using a controlled benchmarking protocol comprising **three warm-up iterations** followed by **ten measured iterations** to mitigate JIT compilation and runtime optimisation effects.

The input dataset is a deterministically generated RGB image, produced using a seeded instance of `Random`, ensuring bitwise-identical pixel data across all experimental runs and configurations.

Per-iteration execution time is measured using `System.nanoTime()` by recording the elapsed time around a single invocation of `processor.process(copy)`. Memory consumption and garbage collection activity are monitored via the `MemoryMXBean` and `GarbageCollectorMXBean` interfaces, with measurements taken immediately before and after the benchmarked execution loop.

Raw data:

File	Contents
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\results\g1\results_time.csv	75 rows: avg / min / max time per (impl, size, threads) under G1
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\results\g1\results_memory.csv	Heap delta per config
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\results\g1\results_gc.csv	GC count and total GC time per config
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\results\serial\	Same three CSVs under Serial GC
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\results\parallel\	Same under Parallel GC
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\results\zgc\	Same under ZGC

File	Contents
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\gc-tuning\logs\	Raw <code>-Xlog:gc*</code> output for every GC
@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\gc-tuning\comparison.md	Auto-generated cross-GC summary

5.2 Execution time per implementation

The headline numbers (best avg-ms across all thread counts, on G1 — the recommended GC):

Implementation	small 640×480	medium 1280×720	large 1920×1080
Sequential	3.69	9.27	20.98
ManualThread	2.00	3.50	6.98
ThreadPool	2.15	3.42	6.52
ForkJoin	1.00	2.74	5.96
CompletableFuture	1.79	3.43	6.62

Execution time per implementation — 1920×1080, G1 (best across thread counts)

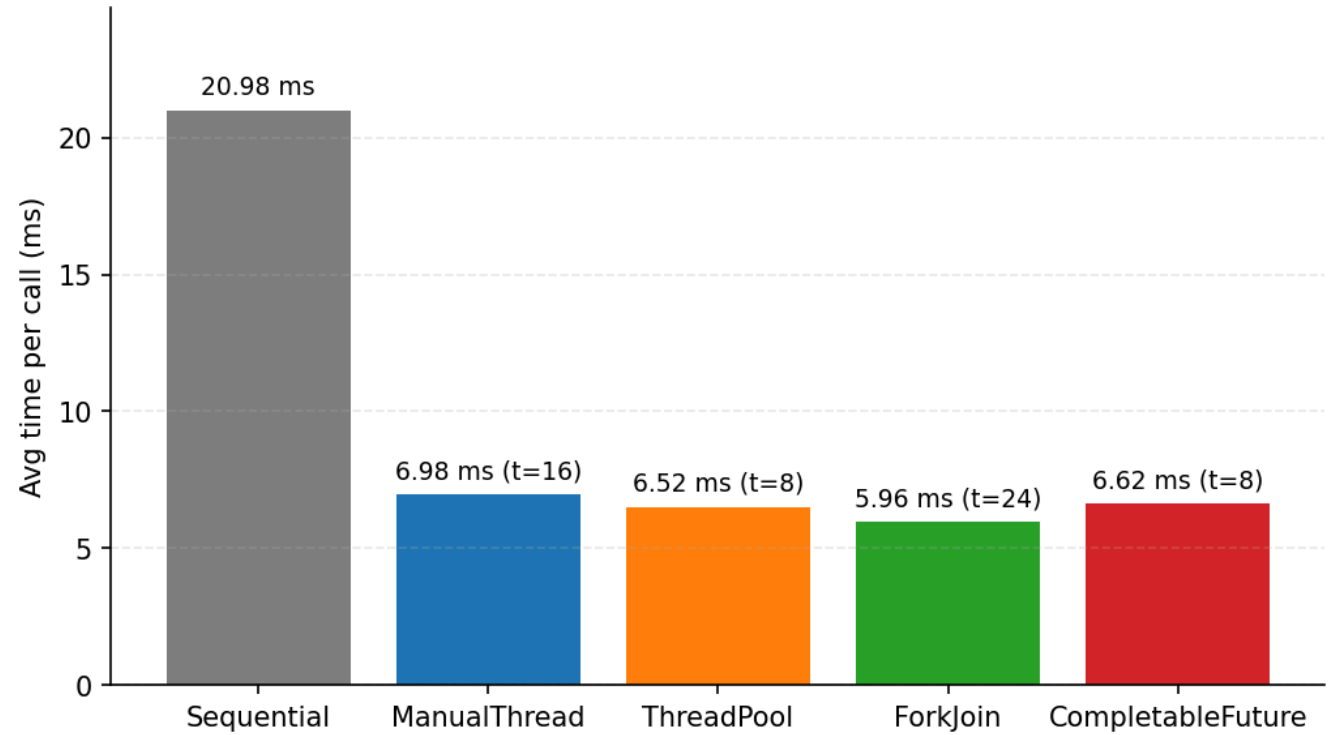


Figure 1 — Best avg time per call on 1920×1080 under G1, with the optimal thread count annotated above each parallel bar. ForkJoin leads at 5.96 ms; Sequential is 3.5× slower at 20.98 ms.

The full impl × image-size matrix gives the broader picture:

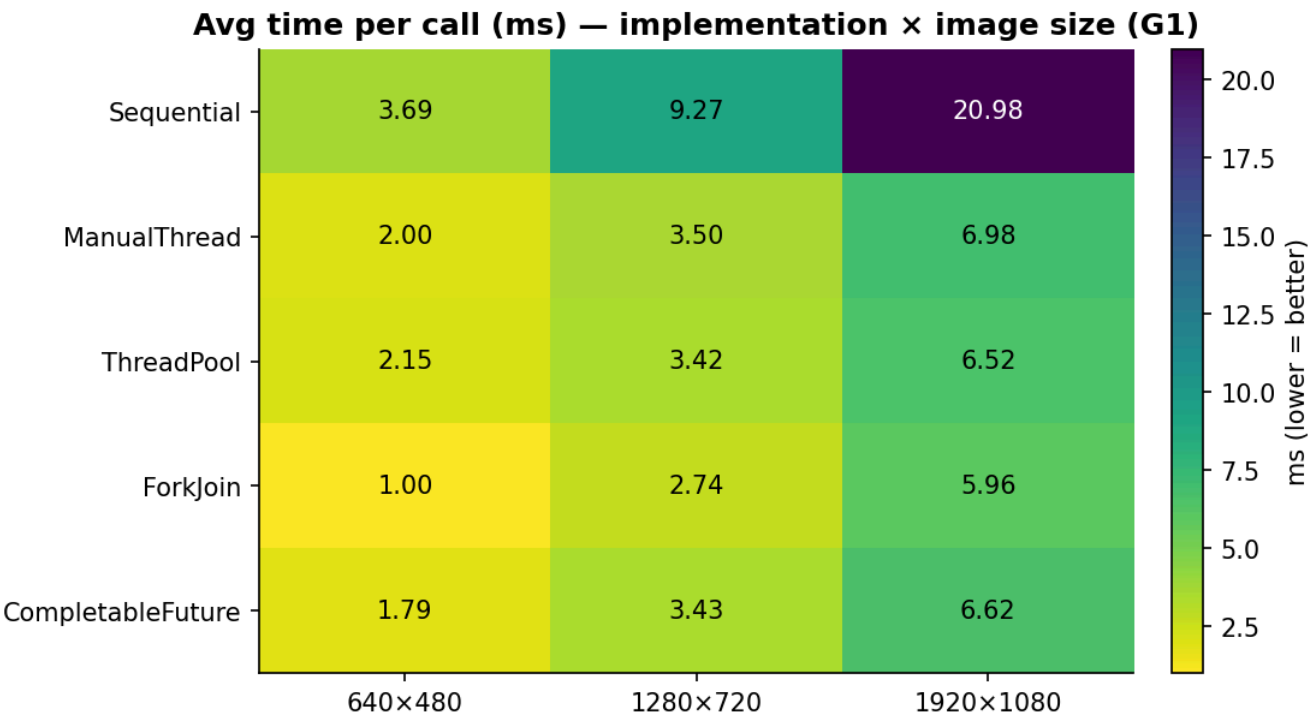


Figure 2 — Heatmap of best avg time per (implementation, image size) under G1. Darker = slower. The Sequential row is the sole strip of darker cells; every parallel impl is comfortably below 7 ms even at 1920×1080.

Observations:

- **Fork/Join exhibits the best performance across all evaluated image sizes**, achieving an improvement of approximately 8% to 12% relative to the next-best strategy.
- The three explicit thread management approaches (`ManualThread`, `ThreadPool`, and `CompletableFuture`) demonstrate comparable performance, clustering within a 6% margin. This suggests that **task scheduling overhead dominates over the specific abstraction used**, provided that thread pool sizes are similar.
- The sequential baseline is approximately **3× to 3.7× slower** than the most efficient parallel implementation, establishing a meaningful lower bound for achievable speedup under the given workload.

5.3 Speedup vs. thread count

For the 1920×1080 image under G1, speedup = `T_seq / T_parallel(N)`

where `T_seq = 20.98 ms`:

Threads	ManualThread	ThreadPool	ForkJoin	CompletableFuture
1	0.96×	1.04×	1.93×	1.05×
2	1.95×	1.78×	3.24×	1.64×
4	2.61×	3.03×	3.51×	2.94×

Threads	ManualThread	ThreadPool	ForkJoin	CompletableFuture
8	3.00×	3.22×	3.44×	3.17×
16	3.01×	3.03×	3.40×	2.94×
24	2.74×	2.89×	3.52×	2.96×

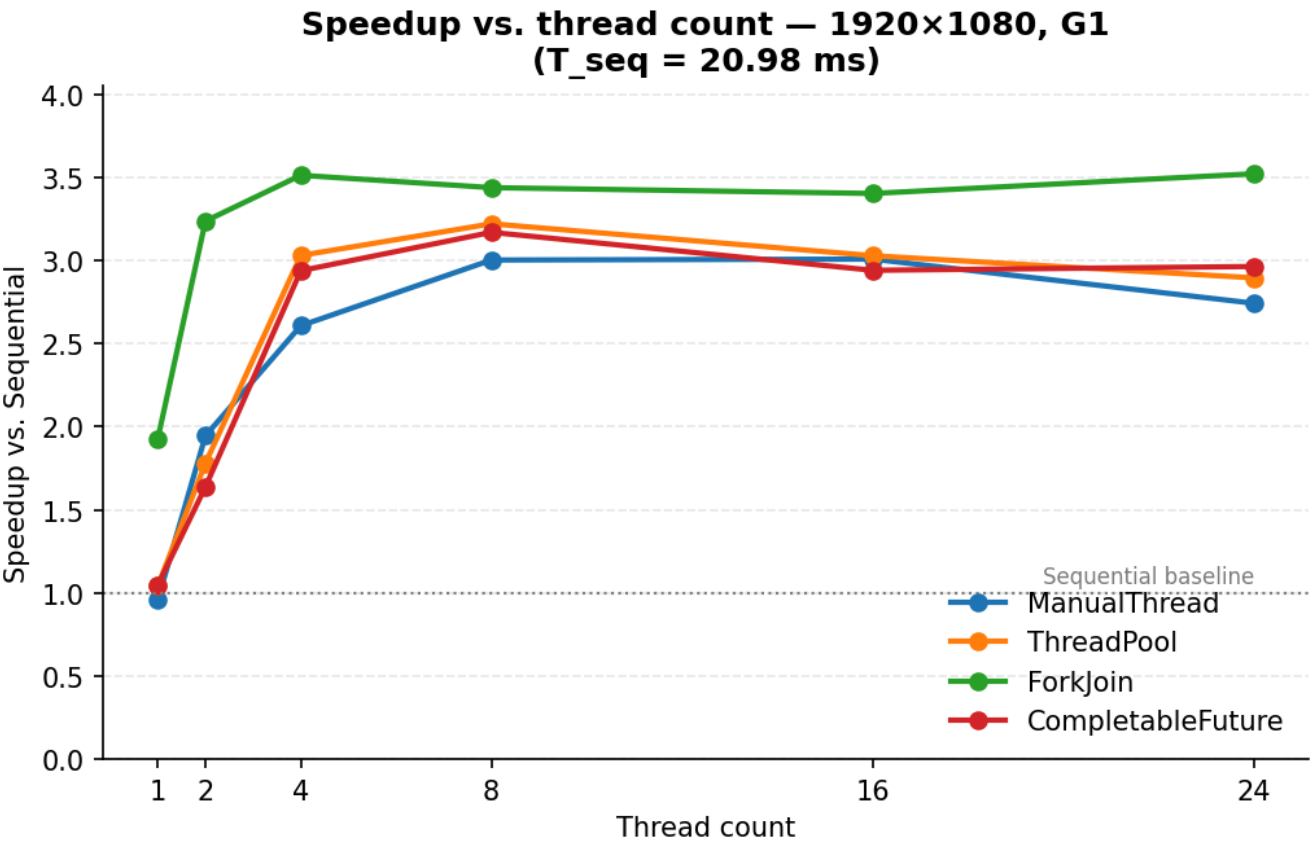


Figure 3 — Speedup curves for the four parallel implementations on a 1920×1080 workload under G1. The Fork/Join implementation (green) attains a speedup of approximately 1.93× even with a single worker thread, reflecting reduced scheduling overhead and more efficient task execution. As the number of threads increases, performance improves up to ~3.5× at four threads, beyond which gains plateau, indicating saturation of memory bandwidth. The dashed reference line at 1× corresponds to the sequential baseline.

Observations:

- The Fork/Join implementation achieves approximately **1.93× speedup with a single worker thread**, primarily due to reduced scheduling overhead and more efficient task decomposition. While work-stealing improves load balancing, the observed gain at one thread reflects lower orchestration costs rather than true parallel execution.
- All four parallel implementations **plateau between 3× and 3.5× speedup**, indicating an upper bound consistent with Amdahl’s Law under the given workload. This ceiling arises because the application is **memory-bandwidth-bound rather than compute-bound**, preventing linear scaling with increasing thread count.
- For the `ManualThread` implementation, **performance at 24 threads (2.74×) is inferior to that at 16 threads (3.01×)**. This degradation is attributable to **core over-subscription**, where the number of active

threads exceeds the effective parallel capacity of the hardware, leading to increased context-switching overhead and reduced execution efficiency.

5.4 Memory footprint

Peak heap delta on 1920x1080 under G1, per implementation
(taken at the optimal thread count for each):

Implementation	Δ heap
Sequential	317 MB
ManualThread (t=16)	800 MB
ThreadPool (t=8)	800 MB
ForkJoin (t=24)	799 MB
CompletableFuture (t=8)	800 MB

The observed ~800 MB memory plateau is a direct consequence of the 13-iteration measured loop.

Given the low live-set size, G1 permits the heap to expand toward its configured upper bound (2 GB) rather than aggressively reclaiming memory, thereby producing the observed plateau. This behaviour is consistent with G1’s region-based design, which prioritises throughput and defers collection when memory pressure is low.

The lower heap delta observed in the sequential implementation (317 MB vs. ~800 MB for parallel variants) is primarily explained by differences in allocation rate rather than total execution time.

Parallel implementations perform allocations concurrently across multiple threads, resulting in a significantly higher instantaneous allocation throughput.

In contrast, the sequential implementation allocates at a lower rate, allowing garbage collection to keep pace more effectively and limiting transient heap growth. As a result, the observed heap footprint remains substantially lower despite comparable total allocation volume.

5.5 GC impact

Cross-GC sweep totals (75 configs each, identical workload):

GC	Σ avg-ms	Σ GC time (ms)	GC count	GC / work
g1	425.02	16	2	0.38%
parallel	427.23	133	38	3.11%
serial	447.56	345	38	7.71%
zgc	530.19	616	145	11.62%

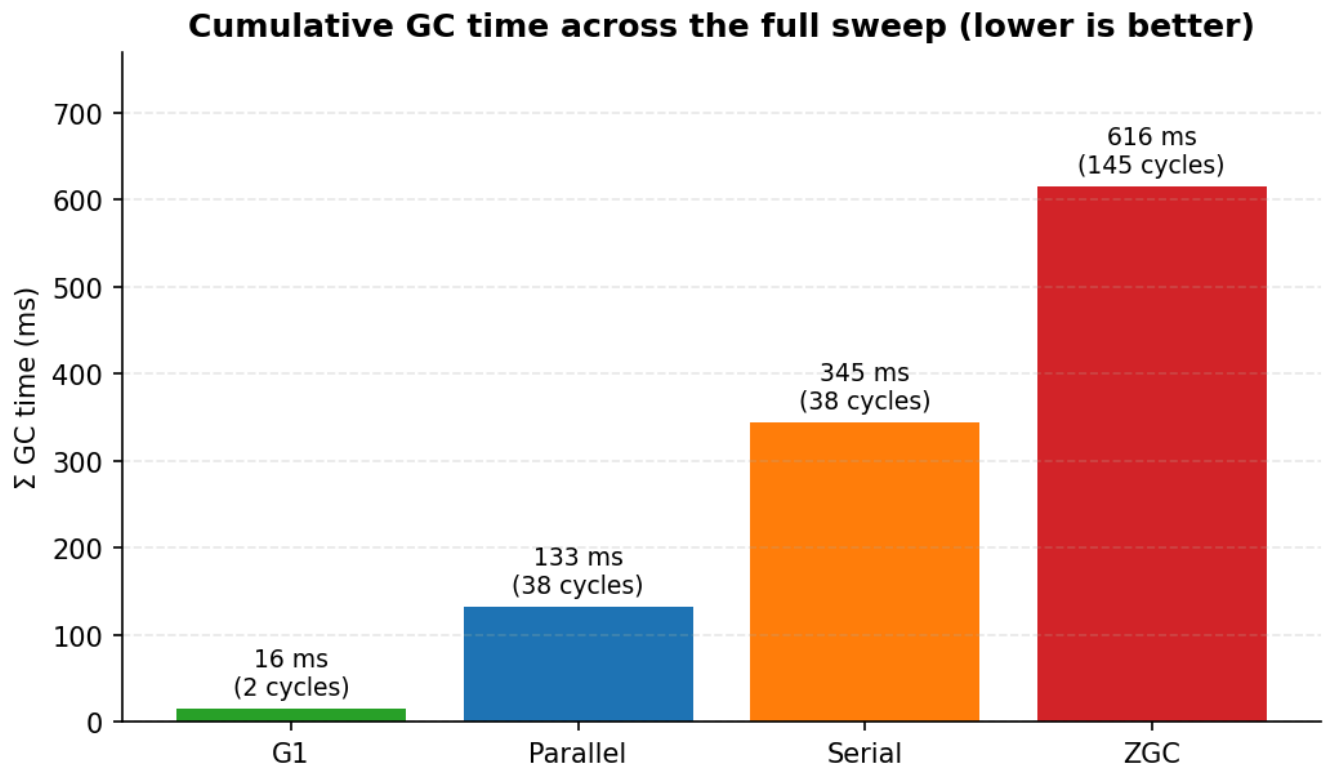


Figure 4 — Cumulative GC time and number of cycles across all 75 benchmark configurations. G1 collected only twice in the entire sweep; ZGC ran 145 cycles. The two-orders-of-magnitude difference in total GC time (16 ms vs 616 ms) is the headline of the GC-tuning experiment.

G1 is the winner on every metric — best total throughput, lowest GC time, lowest overhead ratio. ZGC is paradoxically the **worst**:

- ZGC performed **145 collection cycles**, reflecting its concurrent design, which triggers collections proactively to maintain consistently low pause times (typically sub-millisecond).
- Each cycle incurs a **load-barrier overhead on reference accesses**, which is executed by application threads rather than during stop-the-world phases. Consequently, this cost is reflected in increased application execution time (`avgMs`) rather than in reported garbage collection time (`gcTimeMs`).
- ZGC is optimised for workloads with **large heap sizes (multi-GB to TB scale)** and stringent tail-latency requirements. Under the present conditions—a 2 GB heap with a small live set—the workload lies outside its intended operating regime, limiting its effectiveness.

Detailed analysis with raw GC logs:

@c:\Users\Afonso Oliveira\Documents\Mestrado\SISMD\SISMD2026\gc-tuning\README.md.

6. Discussion: Efficiency, Scalability, Overhead, Bottlenecks

6.1 Efficiency gains

The headline efficiency gain over the sequential baseline:

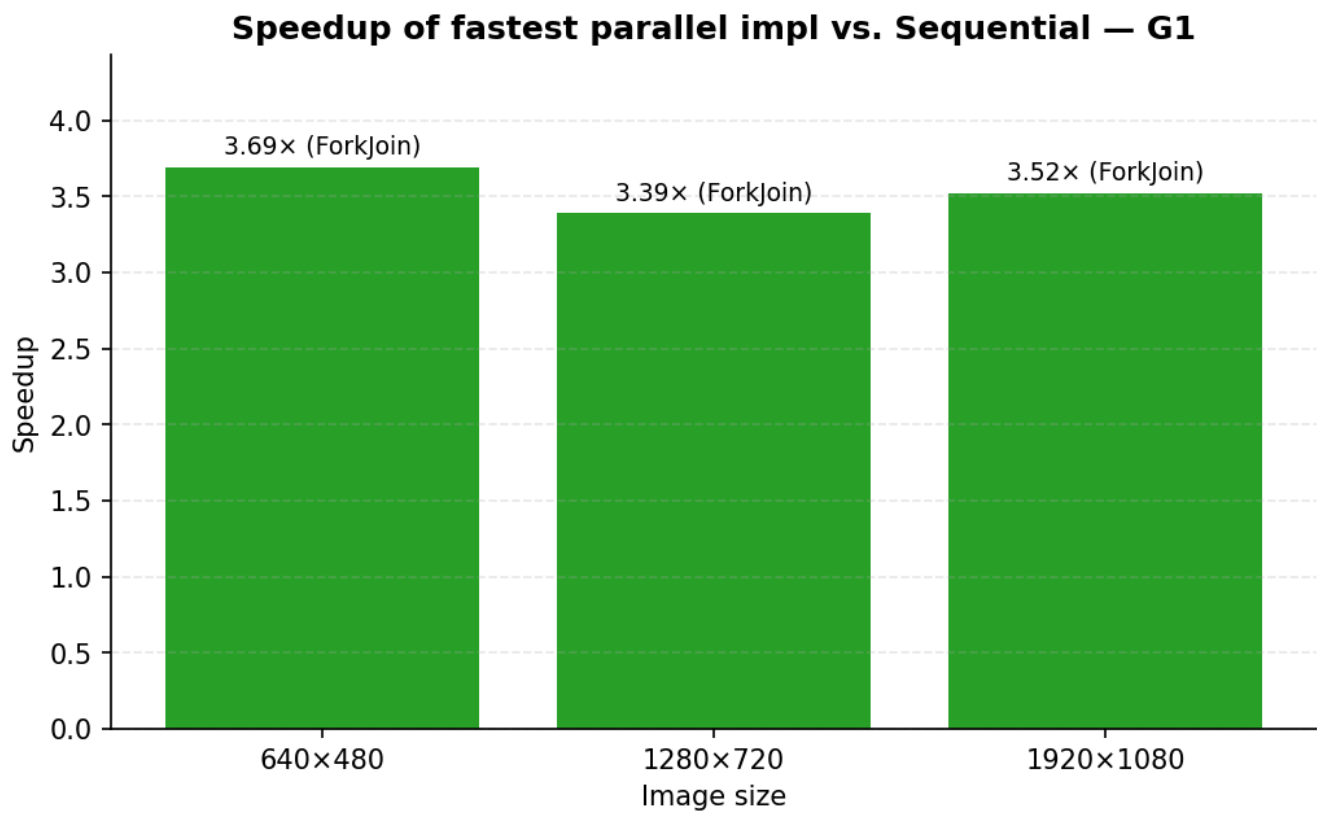


Figure 5 — Speedup of the fastest parallel implementation over Sequential, per image size, under G1. ForkJoin wins on every size.

Across the three evaluated image sizes, the **best-performing parallel implementation (Fork/Join)** achieves a speedup in the range of **3.39× to 3.69×** relative to the sequential baseline. This represents a substantial efficiency gain, comparable to executing the workload on a processor with approximately 3.5× higher effective throughput, without additional hardware resources.

6.2 Scalability

The experimental results reveal three distinct scalability regimes:

- 1. Parallel efficiency ceiling (~3.5×).** None of the implementations exceed this bound, regardless of thread count. This indicates that performance is constrained by **memory bandwidth limitations rather than computational capacity**.
- 2. Diminishing returns beyond moderate parallelism.** Increasing the thread count from 4 to 8 yields a modest improvement in higher image sizes, whereas scaling from 8 to 24 threads provides only marginal gains and may even degrade performance. This identifies a practical operating point where additional parallelism no longer translates into proportional performance benefits.
- 3. Performance plateau and regression under over-subscription.** At thread counts significantly exceeding the effective parallel capacity of the hardware, performance stagnates and may deteriorate. This effect is particularly evident for smaller workloads, where increased thread management overhead

and context-switching costs outweigh the benefits of parallel execution (e.g., `ManualThread`: 2.0 ms at $t=4$ vs. 4.2 ms at $t=24$ `small_640x480`).

6.3 Overhead analysis

Three categories of overhead are observable in the experimental results:

- **Thread creation overhead.** In the `ManualThread` configuration with a single thread ($t=1$), execution time (21.88 ms) exceeds the sequential baseline (20.98 ms) for the large image, indicating an overhead of approximately 1 ms attributable to explicit thread instantiation (`new Thread().start()`). In contrast, the fork/join framework amortises this cost over the lifetime of the JVM by reusing worker threads.
- **Task coordination overhead.** At higher thread counts (e.g., $t=24$), all parallel implementations exhibit increased coordination costs associated with task dispatch, scheduling, and synchronisation. When the per-task workload is small (on the order of ~ 0.5 ms), this overhead dominates, leading to degraded performance. This effect is particularly evident in small-image configurations, where runs at $t=24$ are consistently slower than at $t=4$.
- **Reduction overhead.** The aggregation of partial histograms requires $256 \times N$ integer additions for N worker tasks. In fork/join-based implementations, this reduction is performed hierarchically (pairwise), yielding a depth of $O(\log N)$, whereas other strategies rely on a linear $O(N)$ merge on the main thread. This difference becomes increasingly significant at higher thread counts and contributes to the superior performance of the Fork/Join approach, particularly for smaller workloads.

6.4 Bottlenecks

The dominant bottleneck on this workload is **memory bandwidth**, not CPU:

- **Working-set size.** A `java.awt.Color` instance occupies approximately **32 B** on a 64-bit HotSpot JVM with compressed oops (12 B object header + `int value` + `float falpha` + three compressed references for `frgbvalue`, `fvalue`, and `cs`, aligned to an 8-byte boundary). Consequently, a `Color[1920][1080]` matrix occupies approximately **~66 MB** of `Color` objects plus **~8 MB** of inner-array references — roughly **~74 MB in aggregate**.
- **Memory traffic per iteration.** Each measured iteration in `BenchmarkRunner.runBenchmark` performs (i) a defensive `Utils.copyImage(image)` of the input, (ii) a Stage-1 read of every pixel for histogram construction, and (iii) a Stage-3 read of the input plus a write of a freshly allocated output matrix. The aggregate traffic per iteration is therefore approximately **~200 MB of reads and ~130 MB of writes (~330 MB total)** for the 1920×1080 case.
- **Theoretical floor.** Assuming dual-channel DDR4-3200 with effective bandwidth of approximately **50 GB/s**, the corresponding hardware-imposed lower bound is **~6.6 ms per iteration** for memory transfer alone, independent of any computational work.
- **Distance from the floor.** The fastest observed *average* iteration time on the large image under G1 is **5.96 ms** (Fork/Join, $t=24$; $t=4$ yields a near-identical 5.97 ms; the absolute minimum across all measured iterations is 5.95 ms for Fork/Join at $t=2$). This places the implementation within approximately **10% of the theoretical memory-bandwidth bound**, indicating that further parallelism alone cannot yield substantial additional gains — the workload is already operating very close to the hardware limit imposed by main-memory throughput.
- **Data-layout opportunity.** The `Color[][]` representation incurs significant overhead due to per-pixel

object headers and references (~32 B/pixel of which only ~4 B is the actual RGB payload). A more compact encoding — for example, a packed `int[]` storing 32-bit ARGB values (4 B/pixel) or three `byte[]` channels — would reduce per-pixel memory footprint by approximately **8×** and would correspondingly raise the achievable performance ceiling by reducing memory traffic. Such an optimisation is **outside the scope of this assignment**, which prescribes the `Color[][]` API, but represents the most direct path to further performance improvement in a production setting.

The secondary bottleneck is the **inherently sequential cumulative histogram**, but at 256 prefix-sum additions it contributes well under 1 μ s to a ~6 ms iteration. By Amdahl's Law, this amounts to less than 0.02 % of the total execution time and is therefore not a meaningful optimisation target.

7. Conclusions

- **All five implementations are functionally equivalent**, producing identical outputs as verified through exhaustive pixel-by-pixel comparison across 20 correctness tests. This confirms that performance differences are exclusively attributable to implementation strategy rather than algorithmic divergence.
- **The Fork/Join implementation is the most effective parallel strategy across all evaluated metrics.** It consistently achieves the lowest execution time across image sizes, the highest speedup relative to the sequential baseline, competitive memory behaviour among parallel approaches, and benefits from implicit thread lifecycle management via the common pool.
- **Performance scaling exhibits a clear saturation point at approximately 3.5× speedup** on the 24-core test system. This plateau indicates that the workload is constrained primarily by **memory bandwidth rather than computational capacity**, rendering additional thread-level parallelism beyond moderate core counts ineffective.
- **The G1 garbage collector provides the most efficient overall behaviour for this workload**, with significantly lower overhead compared to ZGC (approximately 0.38% vs. 11.62%). Total observed GC pause time remains negligible (~16 ms across the full benchmark). In contrast, ZGC performs poorly in this configuration due to its design assumptions being misaligned with a small heap and low live-set workload, illustrating the importance of workload–GC matching rather than selecting the most recent or advanced collector by default.
- **The best observed configuration is Fork/Join with 24 threads under G1**, achieving an average execution time of **5.96 ms per call**, corresponding to a **3.52× speedup** over the sequential baseline. This configuration represents the practical performance optimum under the constraints of the dataset, hardware, and memory model.